



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

迭代器模式（Iterator）

一、意图（Intent）

在嵌入式 C 里，迭代器模式的核心不是“做一个对象负责遍历”，而是：

在不暴露容器内部组织方式的前提下，按统一方式依次访问注册项、链表节点、设备表、观察者表或链接段条目。

它控制的是：

- 外部怎样顺序访问元素
- 容器内部怎样存放元素
- 遍历过程中允许哪些操作

所以迭代器模式解决的，是**访问方式统一与存储细节隐藏**的问题。

二、本质 / 动机（Motivation）

在嵌入式工程里，数据集合的内部表示经常不是普通数组，常见形态包括：

- 单链表 / 双链表
- 环形缓冲区
- 静态注册表
- linker section
- 设备实例表

如果客户端代码直接依赖这些内部结构，就会出现几个问题。

1. 遍历代码和存储实现绑死

一旦从数组改成链表、从链表改成链接段表，所有调用方都得跟着改。

2. 容器内部细节泄漏

客户端本来只想“挨个访问元素”，却被迫知道：

- 节点字段叫什么
- 头指针在哪里
- 结束条件怎么判断
- 删除时该怎么安全前进

这会导致上层代码越来越依赖底层布局。

3. 多种遍历需求难以统一

同一批元素往往会有不同访问策略：

- 正向遍历
- 逆向遍历
- 过滤遍历
- 安全删除遍历
- 只读遍历

如果把这些逻辑都塞到容器里，容器会越来越臃肿。

4. 开源内核、RTOS、厂家 SDK 大量默认采用统一遍历接口

像 Linux kernel 的 list 宏、Zephyr 的链表 API、观察者遍历接口、设备注册表遍历，本质上都在做同一件事：

让使用者按统一规则访问元素，而不必直接碰容器内部布局。

这就是迭代器模式在 C 工程里的真实形态。

三、适用性（Applicability）

迭代器模式适合这些条件：

- **容器内部实现可能变化**：数组、链表、注册段都可能替换
- **外部只想访问，不想依赖布局**
- **存在多种遍历策略**：正向、逆向、安全删除、过滤
- **希望统一使用方式**：不同集合用类似方式访问
- **希望降低上层对底层数据结构的耦合**

一句话判断：

如果系统的重点是“怎样稳定访问元素”，而不是“元素底层怎么存”，就该用迭代器思路。

四、结构（Structure）

在嵌入式 C 里，迭代器不一定长成 GoF 图里的对象层次。
更常见的是下面三种形态：

1. 游标结构体 + next/current 接口
2. foreach 宏 + 容器约定
3. 回调式遍历接口

把它抽象成关系，可以写成：

```
Aggregate(集合)
  |
  v
Iterator(游标/遍历规则)
  |
  v
Client 逐项访问元素
```

若把工程层级说清楚，更像是：

```
设备表 / 链表 / 注册段
  |
  v
统一遍历协议
  |
  v
业务层按顺序读元素
```

关键不是外形，而是职责拆分：

- 容器负责保存元素
- 迭代器负责定义访问顺序
- 客户端负责使用访问结果

五、参与者（Participants）

1. Iterator（迭代器 / 游标规则）

在嵌入式 C 里，Iterator 常体现为：

- 当前游标
- 获取当前元素的方法
- 进入下一个元素的方法
- 判断结束的方法

它定义了“怎么走”。

2. ConcreteIterator（具体迭代器）

它知道某个具体集合怎样前进，例如：

- 数组按索引前进
- 链表按 `next` 前进
- 双链表支持反向前进
- 链接段按地址范围前进

3. Aggregate（聚合 / 容器）

容器负责存数据。

在 C 工程里，它可能是：

- 一个链表头
- 一个设备表
- 一个注册段区间
- 一个 observer 集合

4. Client（客户端）

Client 不应该直接依赖容器内部表示。

它最好只知道：

- 如何开始遍历
- 如何拿当前项
- 如何前进
- 何时结束

六、协作（Collaborations）

迭代器模式的典型协作流程是：

1. Client 向集合请求一个遍历入口，或拿到约定好的遍历宏/接口
2. Iterator 定位到第一个有效元素
3. Client 读取当前元素
4. Iterator 前进到下一个元素
5. 重复以上过程直到结束

这里最重要的是：

客户端在“顺序访问元素”，但不需要知道容器内部长什么样。

这让底层从数组改为链表、从普通表改为注册段时，上层代码仍可以保持相对稳定。

七、效果（Consequences）

收益

1. 容器内部结构对外隐藏

客户端不用依赖底层节点布局、头尾指针和边界规则。

2. 遍历逻辑可以独立演化

不同遍历策略可以在不改容器的前提下扩展。

3. 客户端代码更统一

无论底层是链表、数组还是注册段，外部都能用相近方式访问。

4. 便于维护大型注册系统

设备表、命令表、观察者表、驱动表这些场景非常受益。

代价

1. 简单场景下可能显得多一层

如果只是遍历一个很小的局部数组，专门抽一层迭代协议可能过重。

2. 安全语义要单独约定

遍历时是否允许删除当前节点、是否允许并发修改、是否需要加锁，这些都要明确。

3. 某些复杂结构的遍历并不轻松

树、图、带回溯规则的集合，具体迭代器仍然可能很复杂。

八、实现要点（Implementation）

1. 在 C 里不要执着于“类式实现”

很多成熟项目直接使用：

- 宏
- 游标结构
- begin/end 区间
- 回调遍历

只要职责成立，它就是迭代器思路。

2. 先定义遍历语义，再定容器结构

要先说清：

- 起点是什么
- 终点是什么
- 前进规则是什么
- 是否允许遍历中删除
- 是否需要锁保护

3. 明确元素所有权

迭代器只是访问，不等于拥有元素。

遍历时拿到的是：

- 只读指针
- 可写指针
- 临时快照

要事先讲清楚。

4. 注意并发和 ISR 场景

如果集合可能被别的线程、中断或回调同时修改，必须明确安全策略。

5. 对静态注册表尤其有价值

嵌入式系统大量使用：

- 设备实例表
- 命令注册表
- 初始化表
- 观察者集合

这类场景特别适合统一迭代协议。

九、典型应用

下面这些都是很多开源项目和厂家默认组织方式中的高频例子：

1. Linux kernel 链表遍历宏

Linux 内核的 `list_for_each_entry()`、`list_for_each_entry_safe()` 等，本质上就是为 intrusive list 提供统一访问方式。

调用者顺序访问元素，但不直接展开底层链表细节。

2. Zephyr 的 `sys_slist` / `sys_dlist`

Zephyr 提供统一的链表 API，让模块在不暴露内部存储细节的前提下组织和遍历节点。

3. Zephyr zbus 的 channel / observer 遍历

zbus 既支持通道定义，也提供遍历 channel、observer 的相关接口。

这说明在一个消息总线系统里，“如何统一访问注册对象”本身就是重要设计点。

4. 设备模型和静态注册表遍历

很多 RTOS、bootloader、厂家 SDK 会把驱动、命令、初始化项放进静态注册表或链接段，再用统一方式遍历。

这就是迭代器模式在 C 工程里的另一种常见形态。

总结

迭代器模式在嵌入式 C 里的本质，就是把“遍历”从具体存储布局里分离出来，让客户端稳定地访问元素，而不用绑定底层数据结构。